

# GrowthPulse Operations Manual

---

|                |                                          |
|----------------|------------------------------------------|
| Document       | GP-OM-001, GrowthPulse Operations Manual |
| Revision       | 1.0                                      |
| Date           | June 11, 2026                            |
| Status         | Released                                 |
| Classification | Internal, confidential                   |
| Applies to     | Firmware v4.3, Web app v2.22.2           |

## About this manual

This is the procedures manual: how to set up a development machine, build and flash a node, wire and calibrate the bench, deploy the application, run the one-time cloud setups, operate each service console, and recover from every known failure. Every procedure here has been performed on real hardware; the troubleshooting catalog in chapter 11 contains only failures that were actually encountered, with their exact symptoms.

The Engineering Manual explains why these procedures look the way they do. The Technical Reference Manual holds the values (pins, constants, endpoints, schemas) the procedures reference.

### The standing rules, before anything else:

1. Never commit a real token, key, or password. The repository firmware carries a placeholder provisioning token; only private builds carry the real one.
2. Netlify environment variable changes do nothing until you trigger a redeploy.
3. A switch to LoRaWAN must start from a node that is on Wi-Fi and online.
4. Do not factory-reset units during routine testing; use the recovery runbook (chapter 10) instead.
5. Never run the simulator against the same device id as live hardware.

---

## 1. Service access inventory

---

The system runs on five external services plus GitHub. One account of each exists; credentials live in the team password manager, never in documents.

| Service          | What lives there                                                                | Console                                                                         |
|------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| GitHub           | The repository; pushing main deploys production                                 | github.com/BryanPuckettGH/growthpulse-app                                       |
| Netlify          | Hosting, build pipeline, the nine functions, all server-side secrets            | app.netlify.com (site: growthpulsecloud.com)                                    |
| Supabase         | Auth users, device_registry, devices, gateways, lorawan_devices                 | supabase.com dashboard, SQL Editor                                              |
| Losant           | The device-cloud application: devices, access keys, API tokens, alert workflows | console.losant.com                                                              |
| The Things Stack | The growthpulse application, end devices, gateways, the uplink webhook          | nam1.cloud.thethings.network/console (login routes through eu1; that is normal) |

Secrets exist in exactly three places: the gitignored `.env`, Netlify's environment store, and the private firmware build. If a secret is ever pasted into a chat, terminal screenshot, or document, rotate it.

## 2. Development environment setup

### 2.1 Web application (any machine)

1. Install Node.js LTS (nodejs.org).
2. `git clone https://github.com/BryanPuckettGH/growthpulse-app.git && cd growthpulse-app && npm install`
3. Create `.env` in the project root with the variable names from Technical Reference section 4.10; values come from the team's service dashboards, never from documents.
4. `npm run dev` opens `http://localhost:5173`.

Two local-development facts: `VITE_`-prefixed variables compile into the public bundle, so never put a secret behind a `VITE_` name; and `npm run dev` runs Vite only, so the serverless functions do not execute and claimed devices sit at "waiting for first reading" locally. To exercise the full pipeline use the deployed site or `netlify dev` (Netlify CLI), which serves the functions too.

### 2.2 Firmware toolchain, from zero

1. **Arduino IDE 2.x** from [arduino.cc/en/software](https://arduino.cc/en/software).
2. **USB driver.** The board talks USB through a CP2102 bridge; without the Silicon Labs CP210x VCP driver it never appears as a serial port. macOS: `/dev/cu.usbserial-XXXX`; Windows: `COMn`.

3. **Board support.** Settings, Additional boards manager URLs, add `https://espressif.github.io/arduino-esp32/package_esp32_index.json` ; Boards Manager, install **esp32 by Espressif Systems**; select **"Heltec WiFi LoRa 32(V3)"** (the Heltec definition carries the correct flash and pin configuration; a generic ESP32-S3 profile does not).
  4. **Libraries** (Library Manager, exact names): WiFiManager (tzapu), OneWire, DallasTemperature, DHT sensor library (Adafruit), Losant Arduino MQTT Client, ArduinoJson, U8g2, RadioLib (jgromes).
  5. **Partition scheme.** Tools, Partition Scheme: a larger-app / "Huge App" layout. The combined image overflows the default slot; this is mandatory.
- 

## 3. Build, flash, and deploy procedures

---

### 3.1 Flashing the production image

1. Open `GP_Combined/GP_Combined.ino` (the private copy with the real provisioning token for units that must self-provision; the repo copy carries a placeholder).
2. Board "Heltec WiFi LoRa 32(V3)", Huge App partition scheme, pick the port, Upload.
3. If the upload hangs at "Connecting..." or dies mid-write ("the chip stopped responding"): set Upload Speed to 115200 first; if needed, force download mode by holding PRG, tapping RST, holding PRG one more second, then uploading. Marginal cables are the usual culprit.
4. Watch the OLED: the self-test screen shows the firmware version and per-sensor OK/"--", then the pairing screen shows the version under the link chip. The on-screen version is the proof the reflash took.
5. Open Serial Monitor at 115200 for the boot banner, pairing code, and telemetry lines.

**Both copies rule:** every firmware change is made to the working copy and the repository copy, versions bumped together, and the CHANGELOG updated. Verify the repo copy still carries the placeholder token before committing.

### 3.2 Deploying the web application

1. Commit and push to `main`. Netlify builds automatically ( `npm run build` , publish `dist/` , bundle `netlify/functions/` with esbuild, per `netlify.toml` ).
2. If the push is rejected by a stale lock: `rm -f .git/index.lock` and retry. If GitHub returns a server-side `commit_refs` error, the commit is safe locally; just `git push` again.
3. Confirm the deploy in Netlify, Deploys. The Settings screen's version stamp confirms what is live.

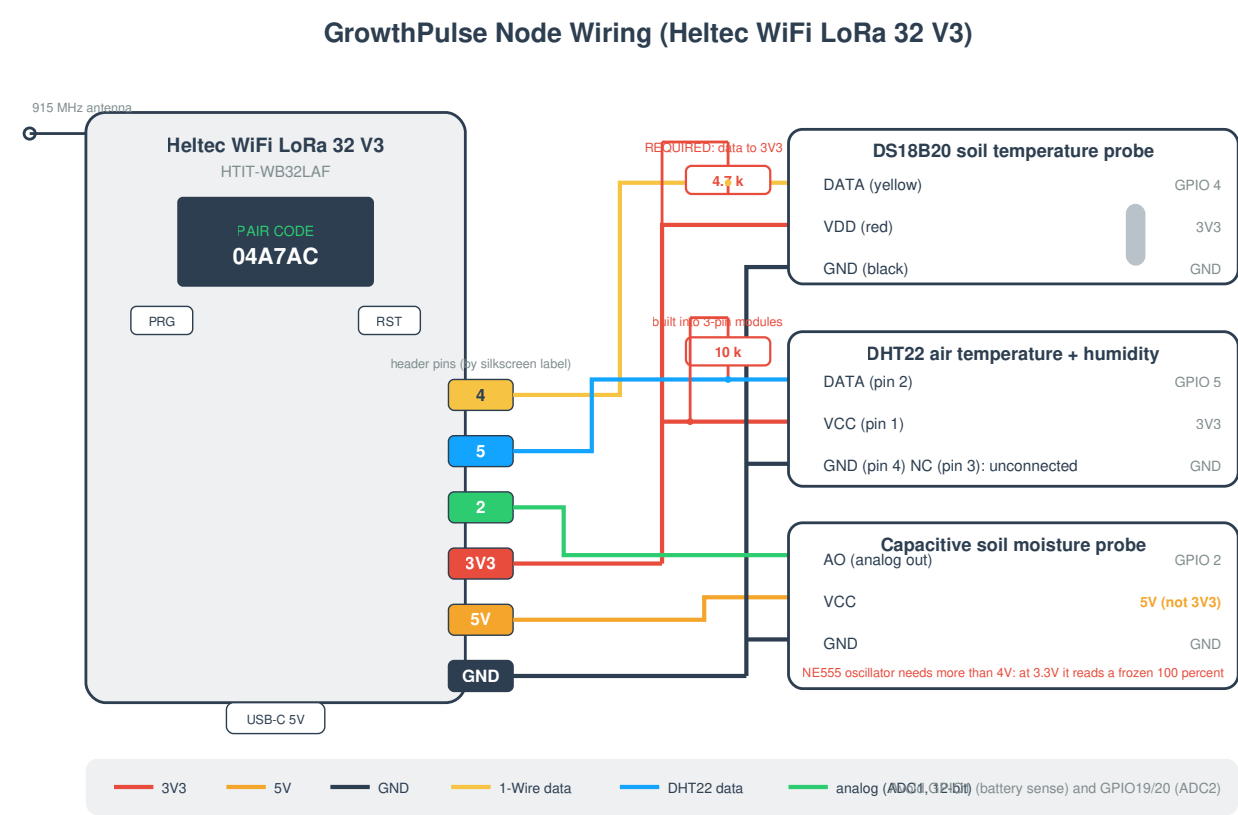
### 3.3 Changing environment variables

Netlify, Site configuration, Environment variables. Add or edit, mark tokens as secret, then **Deploys, Trigger deploy**. Variables do not apply to the running site without the redeploy; this has caused more confusion than any other operational detail.

## 4. Bench procedures

### 4.1 Wiring a node

Work from the wiring diagram and the quick reference:



| Sensor                                     | VCC | GND | Signal (silkscreen label) | Pull-up                                               |
|--------------------------------------------|-----|-----|---------------------------|-------------------------------------------------------|
| DS18B20 soil temp (red/black/yellow)       | 3V3 | GND | 4                         | 4.7k from data to 3V3, required                       |
| DHT22 air (bare 4-pin: VCC, DATA, NC, GND) | 3V3 | GND | 5                         | 10k from data to VCC (3-pin modules have it built in) |
| Capacitive soil moisture                   | 5V  | GND | 2                         | none                                                  |

Finding the pins on the silkscreen (HTIT-WB32LAF): the top header reads 7 6 5 4 3 2 1 ... from the left and carries the signal pins plus two 3V3 pins at the right end; the bottom header's right end reads ... 5V GND . Match labels; do not count GPIOs. The OLED uses 17/18/21/36 internally; wire nothing to those.

Power: USB-C from any charger, or a bench supply at 5.0V with a ~1.0A current limit into the 5v and GND pins (the limit is headroom; too low causes brownout resets mid-Wi-Fi). Never feed 5V into a 3V3 pin.

## 4.2 Power-on check

1. Serial Monitor at 115200: real numbers, not -196.6 and not nan.
2. Soil and air temperature near room temperature; humidity believable.
3. The boot self-test screen shows OK on all three lines.
4. The app shows the same values within a few seconds of the unit going online.

If soil temperature is -196.6: the DS18B20 pull-up. If air values are nan: the DHT22 data pin or its pull-up. If moisture is stuck at 100 percent with raw ~120: the probe is on 3.3V; move it to 5V.

## 4.3 Soil probe calibration

1. With the unit running, watch raw= on the serial line.
2. Probe in dry air: note the number (typically near 3600). That is dryValue .
3. Probe in a glass of water up to its line (never past the electronics): note the number (typically near 1300). That is wetValue .
4. Edit the two constants at the top of the firmware, reflash both copies.

## 4.4 Diagnostics

- **ADC pin in doubt:** flash firmware/diagnostics/ADC\_Scanner , jumper the pin to 3V3 (expect ~4095) and GND (expect ~0). Judge a pin by its response to a known voltage, not by the presence of a number; floating pins read plausible-looking noise.

- **1-Wire bus in doubt:** flash `firmware/diagnostics/OneWire_Scanner`. A real DS18B20 prints an address starting 0x28. Nothing on any pin: the probe never reaches the bus (power, contact, pull-up). Random addresses: line noise, which at least proves the pin and code are alive.
- **Cloud pipeline without hardware:** `cd growthpulse-simulator && node simulator.mjs`. Never simultaneously with the real board on the same device id.

---

## 5. Provisioning operations

---

### 5.1 What happens automatically

A new board flashed with the production image needs **zero manual cloud work**: on first Wi-Fi boot it posts its pairing code to `provision-device`, which creates its Losant device (full attribute schema), registers the code in `device_registry`, and returns a device-scoped key the board saves to flash. The customer claims it with the code on the screen. A factory-reset or reflashed board reclaims the same identity because the code is chip-derived.

### 5.2 One-time backend setup (already done; rebuild reference)

If the backend ever has to be rebuilt from scratch:

1. **Losant:** create the application; create three API tokens (read-only `all.Application.read`; full-access for commands; provisioning with `devices.*` and `applicationKeys.*`). Build the alert workflows (chapter 8).
2. **Supabase:** create the project; run, in the SQL Editor, `growthpulse-supabase-schema.sql`, `growthpulse-devices-schema.sql`, `growthpulse-devices-photo.sql`, `growthpulse-gateways-schema.sql`, then `growthpulse-lorawan-routes-schema-v2.sql`. Copy the URL, anon key, and service-role key.
3. **TTS:** create the `growthpulse` application; create an API key with end-device and gateway write rights; add the uplink webhook (section 7.3).
4. **Netlify:** set every variable in Technical Reference 4.10, then trigger a deploy.
5. **Firmware:** put the chosen `PROVISION_TOKEN` value into the private build; it must equal the Netlify variable.

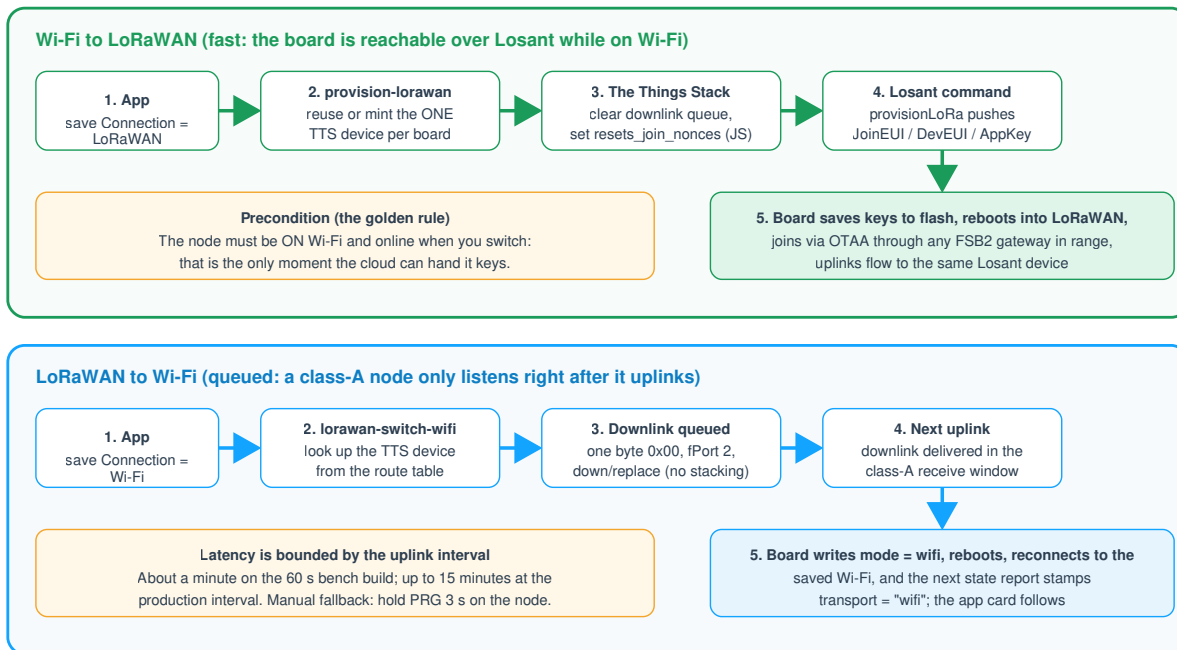
### 5.3 Verifying a board's self-provision

Netlify, Functions, provision-device, logs: a 200 with the device id on first boot. Losant: a new "GrowthPulse CODE" device receiving state. Serial: "First-time setup / registering device" then "Provisioned. Losant device id: ...". Error meanings: 401, the firmware token and Netlify `PROVISION_TOKEN` do not match; 502 device create, the Losant provisioning token lacks scopes; 502 registry insert, the Supabase service-role key is wrong or missing.

## 5.4 Switching a node to LoRaWAN (and back)

From the app only, Devices, edit, Connection: Wi-Fi to LoRaWAN requires the node online on Wi-Fi (the backend pushes keys through a Losant command; the node saves them and reboots into LoRaWAN). LoRaWAN to Wi-Fi queues the 0x00 downlink, which lands on the node's next uplink, up to one uplink interval later (about a minute on the current bench build; up to 15 minutes at the production interval). The plant stays the same plant in the app throughout.

### Transport Switching, Both Directions



The plant stays the same plant in the app throughout: both transports write to the same Losant device.

## 6. Gateway bring-up (ThinkNode G1)

One-time per gateway, before it ships or before bench use.

1. **Register on TTS** (the app's Add-a-gateway does this automatically for customers; manually: Console, Gateways, Register gateway; EUI from the gateway label's `EUI:` line; gateway id convention `eui-eui lowercase`; frequency plan **United States 902-928 MHz, FSB 2 (used by TTN)**).
2. **Point the gateway at TTS.** In the G1's web UI (join its AP or LAN, browse 192.168.1.1, login root/root): LoRaWAN, LoRa Gateway, Mode = Packet Forward; Server Address `nam1.cloud.thethings.network`, ports 1700/1700; Channel Plan US915 **sub-band 2**. Save, apply, restart.
3. **Verify:** the TTS console gateway page shows Connected and Live data shows traffic.

The FSB2 step is the one that matters: the G1 ships on FSB1, and on FSB1 the gateway cannot hear GrowthPulse nodes at all (joins fail -1116 while the gateway looks healthy). Old tutorials pointing at `router.us.thethings.network` are the legacy V2 network; do not use them.

---

## 7. Cloud console operations

---

### 7.1 Losant operations

- **Alert workflow:** Workflows, create; Device State trigger; conditional, for example `{{ data.soilMoisturePercent }} < 25`; wire the conditional's **true output forward** into the Email/SMS node (backwards wiring silently does nothing). Remember the one-email-per-minute rate limit when testing.
- **Token hygiene:** the read-only, command, and provisioning tokens are separate on purpose; never widen the read token's scope.
- **Inspecting a device:** Devices, select, the attributes view shows the latest composite state; the device log shows accepted and rejected reports.

### 7.2 Supabase operations

- **Migrations:** SQL Editor, New query, paste the schema file, Run. All schema files are in `docs/` and are idempotent (safe to re-run).
- **Registry inspection:** Table Editor, `device_registry`. One row per provisioned unit. Manual seeding is only needed for legacy units; self-provisioning writes new rows itself.
- **Releasing a claim manually:** delete the unit's row in `devices` (normally the app's Remove does this).
- **Routing inspection:** `lorawan_devices` should hold **exactly one row per board**. Multiple rows for one Losant id predate the v2 migration; re-run `growthpulse-lorawan-routes-schema-v2.sql`.

### 7.3 The Things Stack operations

- **Webhook (must exist, exactly one):** Application growthpulse, Integrations, Webhooks: JSON format, base URL `https://growthpulsecloud.com/.netlify/functions/lorawan-uplink`, Uplink message enabled with an empty path, header `X-Webhook-Token` equal to the Netlify `LORAWAN_WEBHOOK_TOKEN`. A missing webhook alone explains an app card stuck on Wi-Fi.
- **Webhook health:** the webhook page shows a status banner. TTS auto-deactivates a webhook after repeated failures; a deactivated webhook with empty Netlify logs means the function was rejecting deliveries (fix the cause, then re-enable).



- **Clearing a stuck downlink queue:** the backend does this at every provision. Manually: `POST https://nam1.cloud.thethings.network/api/v3/as/applications/growthpulse/devices/<device>/down/replace` with body `{"downlinks":[]}` and the TTS API key as a Bearer token.
- **Setting resets\_join\_nonces by hand:** it is a **Join Server** field; the console toggle is unreliable. `PUT ../api/v3/js/applications/growthpulse/devices/<device>` with `{"end_device":{"ids":{"device_id":"<device>", "dev_eui":"<DEVEUI>", "join_eui":"0000000000000000"}, "resets_join_nonces":true, "paths":["resets_join_nonces"]}}`. A response echoing `"resets_join_nonces": true` confirms it. Sending it to the `ns/` endpoint fails with "forbidden path(s) in field mask"; that error naming a field tells you which server owns it.
- **Deleting orphan devices:** End devices; `gp-...` entries showing "Never" activity are pre-idempotency orphans and safe to delete.
- **Live debugging:** keep the **gateway's** Live data open while a node transmits. Join request visible there: the radio side works, suspect keys or sub-band. Nothing there: radio, range, or gateway configuration.

## 7.4 Spectator/teammate setup

A teammate brings up the entire LoRaWAN chain through the app, with no TTS console work. They flash `firmware/GP_Combined/GP_Combined.ino`, claim the node over Wi-Fi, point their own gateway at the TTS server (Packet Forwarder -> `nam1.cloud.thethings.network:1700`, US915 FSB2) and add it in the app (which auto-registers it on TTS), then switch the node to LoRaWAN in the app. `provision-lorawan` generates the OTAA keys, creates the end device on TTS, stores the route, and pushes the keys to the board, which reboots into LoRaWAN and joins through the gateway. No manual device, keys, payload formatter, webhook, or routing row is needed. See the GrowthPulse LoRaWAN Teammate Checklist.

# 8. Recovery runbooks

## 8.1 Node misbehaving (stuck, bouncing, or showing the wrong link)

The clean reset sequence; it avoids factory-reset identity churn:

1. **Get the node onto Wi-Fi and online.** Stuck on LoRaWAN: hold PRG 3 seconds (graceful, keeps credentials). Wi-Fi wiped: redo the hotspot setup; the board reclaims its same Losant device automatically.
2. **Confirm the app card shows Wi-Fi and online.** This proves board and card are bound to the same Losant device.

3. **Switch to LoRaWAN in the app once.** Idempotent provisioning reuses the one TTS device, clears its downlink queue, sets resets\_join\_nonces, pushes the keys; the node joins on the first try and holds.
4. **To return, switch to Wi-Fi in the app.** The downlink lands within one uplink interval.

## 8.2 Symptom-specific recoveries

| Situation                                                  | Action                                                                                                                                                                                                                                     |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Node bounces Wi-Fi/LoRaWAN repeatedly                      | A stale downlink or stale command is looping it. Clear the TTS downlink queue (7.3), then run 8.1. Firmware v4.1+ already ignores stale commands for 12 s after connect                                                                    |
| Node on LoRaWAN, app cannot reach it, downlink not landing | PRG 3-second hold, then 8.1                                                                                                                                                                                                                |
| Card stuck on Wi-Fi after a LoRaWAN switch                 | Check the TTS webhook banner (deactivated?) and the Netlify lorawan-uplink log for 404/502 outcomes; a 502 "rejected state" self-heals after the node's next Wi-Fi check-in re-provisions the attribute schema, then re-enable the webhook |
| Joins slow after a reflash                                 | Verify resets_join_nonces is true on the Join Server (7.3); v2.22.2+ sets it at every provision                                                                                                                                            |
| Unit must go to a new owner                                | App, Factory reset (online), or PRG 10-second hold (offline). Both end at the setup hotspot                                                                                                                                                |
| Wi-Fi changed (new router/password)                        | PRG 10-second hold, redo setup. Account data is unaffected                                                                                                                                                                                 |
| Commit blocked by .git/index.lock                          | <code>rm -f .git/index.lock</code> , retry                                                                                                                                                                                                 |
| Function behaves stale after env change                    | Trigger a Netlify redeploy                                                                                                                                                                                                                 |

## 9. Pre-production checklist

Before shipping units beyond the bench:

1. Raise `LORA_UPLINK_MS` toward 15 minutes (TTN fair-use), accepting the switch-back latency; update the User Manual wording accordingly.
2. Rotate every service token and the provisioning token; confirm no secret has ever entered git history.
3. Pin the TLS certificate (or root CA) in the firmware provisioning call, replacing `setInsecure()`.

4. Add rate limiting to `provision-device` ; consider per-batch tokens.
5. Confirm gateway images ship pre-pointed at the GrowthPulse network server on FSB2.
6. Re-verify both firmware copies are balanced and the repo copy carries the placeholder token.
7. Run the full round trip on hardware: claim, Wi-Fi telemetry, switch to LoRaWAN, verified uplinks with true RSSI/SNR on the card, switch back, factory reset, re-claim.

## 10. Troubleshooting catalog

Every entry was actually encountered. Symptoms are exact.

### 10.1 Hardware and sensors

| Symptom                                                    | Cause                                                                  | Fix                                                                      |
|------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Soil temperature -196.6 F                                  | DS18B20 absent or missing/<br>misplaced 4.7k pull-up (-127 C sentinel) | Bridge data and 3V3 with 4.7k;<br>reseat the probe                       |
| Air temp/humidity nan or null                              | DHT22 absent or data wire loose                                        | Check pin 5 and the pull-up                                              |
| Moisture pinned at 100 percent,<br>raw ~120, ignores water | Probe powered at 3.3V; NE555 dead<br>below ~4V (LED still lights)      | Power the probe from 5V                                                  |
| Soil raw flat 0 on GPIO1                                   | GPIO1 is the battery-sense net                                         | Use GPIO2 (firmware default)                                             |
| Floating ADC pin "reads<br>something"                      | ADC noise on an undriven pin                                           | Judge by the 3V3/GND jumper test,<br>not by the presence of a number     |
| 1-Wire scanner prints garbage<br>addresses                 | Contact bounce parsed as devices<br>(real addresses start 0x28)        | Fix the physical bus; the pin and<br>code are proven alive               |
| Battery reads 0 percent / AC on<br>battery                 | Sense control polarity (pre-v3.7<br>firmware)                          | Flash current firmware                                                   |
| Battery percent jumps when<br>charger connects             | Terminal-voltage inflation (pre-v3.8)                                  | Flash current firmware; percent now<br>creeps 1 percent per reading      |
| Phantom climbing percent,<br>USB, no battery               | No battery-detect pin; sense line<br>pinned high                       | Current firmware reports AC; the<br>VBUS divider mod is the complete fix |

## 10.2 Wi-Fi setup and flashing

| Symptom                                        | Cause                                            | Fix                                                                   |
|------------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------|
| Phone hangs "connecting..." to the hotspot     | AP+STA channel hopping (pre-fix firmware)        | Current firmware disables auto-reconnect and modem sleep during setup |
| Captive portal never pops                      | Phone portal detection suppressed                | Browse to 192.168.4.1; turning off cellular data helps                |
| Portal vanished mid-setup                      | Old 180 s timeout                                | Current timeout is 600 s                                              |
| Board joins old Wi-Fi instead of opening setup | Saved credentials still valid                    | PRG 10 s (or PRG at power-on) to wipe                                 |
| Upload dies, "chip stopped responding"         | 921600 baud over a marginal cable                | 115200; PRG+RST download mode; better cable                           |
| No serial port at all                          | CP210x driver missing                            | Install the Silicon Labs VCP driver                                   |
| Blank OLED on custom code                      | Vext not driven LOW, or generic U8g2 constructor | Power the rail, use the V3 pin constructor                            |
| Brownout resets during flash/Wi-Fi             | Weak cable or supply current limit               | Better cable; 1A bench limit                                          |

## 10.3 Cloud and app

| Symptom                                      | Cause                                               | Fix                                                              |
|----------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------|
| Email alert "did not fire"                   | One-per-minute Losant rate limit, or spam folder    | Space test readings a minute apart                               |
| 404 "Application not found" from Losant REST | Wrong application id                                | The app id comes from the dashboard URL, not from inside a token |
| Claimed device stuck "waiting" locally       | <code>npm run dev</code> does not run the functions | Use the deployed site or <code>netlify dev</code>                |
| Pairing code rejected                        | Typo, or the code is not in the registry            | Read it off the OLED; verify the registry row                    |
| "Already claimed by another account"         | claim_code UNIQUE constraint doing its job          | The other account must remove it first                           |
| Report tab never opens                       | Pop-up blocked                                      | Allow pop-ups once, or use Download PDF                          |
| Gateway QR camera will not open or scan      | Pre-v2.22.1 scanner restart loop                    | Deploy current app; the camera now starts once and stays up      |

## 10.4 LoRaWAN

| Symptom                                              | Cause                                                         | Fix                                                                                 |
|------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------|
| radio.begin() returns -707/-706                      | TCXO voltage zero or wrong                                    | RadioLib default 1.6V on the V3                                                     |
| radio.begin() returns -2 (chip not found)            | Wrong NSS/BUSY/RST pins or SPI                                | 8/14/12/13, SCK 9, MISO 11, MOSI 10                                                 |
| Join -1116, gateway "hears nothing"                  | Sub-band mismatch (gateway FSB1 vs TTN FSB2)                  | Set the gateway channel plan to FSB2                                                |
| Join -1116 only after reflashes                      | DevNonce catch-up                                             | resets_join_nonces on the <b>Join Server</b> ; current backend sets it at provision |
| First uplink -1101 (session not active)              | beginOTAA without activateOTAA                                | Current firmware; the activate retry loop                                           |
| TX "succeeds," nothing ever received                 | DIO2 RF switch not enabled                                    | setDio2AsRfSwitch(true)                                                             |
| Downlink rejected, no_device_session                 | Aimed at an orphan TTS device the node never joined           | Idempotent provisioning + the cleanup migration; delete orphans                     |
| Node kicked back to Wi-Fi right after joining        | Stale 0x00 in the downlink queue                              | Clear the queue (7.3); current backend clears at provision                          |
| Card stuck on Wi-Fi, webhook "healthy," empty logs   | Webhook auto-deactivated after Losant 502s (attribute schema) | Read the banner; schema self-heals at next provision; re-enable                     |
| Switch-to-Wi-Fi "takes forever"                      | Class-A physics: delivery rides the next uplink               | Bounded by the uplink interval (60 s bench, 15 min production)                      |
| Join request in gateway data, nothing in device data | DevEUI/JoinEUI/AppKey mismatch, or FSB mismatch               | Keys must match the Join Server exactly; both sides FSB2                            |

GrowthPulse Engineering. Internal and confidential. Keep credentials out of documents and repositories.